```
import { useMutation, useQuery } from "@tanstack/react-query";
import { createFileRoute, Link, useSearch } from "@tanstack/react-router";
import { useServerFn } from "@tanstack/react-start";
import {
  ArrowLeft,
  ArrowRight,
  CalendarDays,
  Check,
  CheckCircle2,
  CircleAlert,
  Clock,
  History,
  Info,
  Loader2,
  Pencil,
  Search,
  X,
} from "lucide-react";
import { useEffect, useId, useMemo, useState } from "react";
import { z } from "zod";

import {
  BookingCalendar,
  FullyBookedLegend,
  TimeSlotPicker,
} from "@/components/site/booking-schedule";
import { CopyReferenceButton } from
"@/components/site/copy-reference-button";
import { SiteFooter } from "@/components/site/site-footer";
import { SiteHeader } from "@/components/site/site-header";
import { TermsAgreement } from "@/components/site/terms-agreement";
import { TurnstileChallenge } from "@/components/site/turnstile-challenge";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import {
```

```
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { supabase } from "@/integrations/supabase/client";
import {
  type Availability,
  computeAvailableDates,
  computeAvailableSlots,
  computeFullyBookedDates,
} from "@/lib/availability";
import {
  BOOKING_DRAFT_LIFETIME_MINUTES,
  clearBookingDraft,
  loadBookingDraft,
  saveBookingDraft,
} from "@/lib/booking-draft";
import { createBooking, getAvailability, getRescheduleDetails } from
"@/lib/booking.functions";
import {
  DEFAULT_BOOKING_TERMS,
  DEFAULT_BOOKING_HOURS,
  PHONE_VALIDATION_MESSAGE,
  MAX_BOOKING_SERVICE_SELECTIONS,
  bookingDurationOverflowMinutes,
  formatDateLong,
  formatPHP,
  formatTime,
  formatNamePartInput,
  isValidCalendarDate,
  normalizePhilippineMobile,
```

```
  sanitizePhilippineMobileInput,
} from "@/lib/shop";
import { cn, randomId } from "@/lib/utils";

const searchSchema = z.object({
  date: z.string().refine(isValidCalendarDate, "Enter a valid appointment
date.").optional(),
  startTime: z
    .string()
    .regex(/^\d{2}:\d{2}$/, "Enter a valid appointment start time.")
    .optional(),
  serviceId: z.string().uuid().optional(),
  reschedule: z
    .string()
    .regex(/^FRM-[A-Z0-9]{6}$/)
    .optional(),
  phone: z
    .string()
    .trim()
    .refine(
      (value) => normalizePhilippineMobile(value) !== null,
      "Enter a valid Philippine mobile number.",
    )
    .transform((value) => normalizePhilippineMobile(value)!)
    .optional(),
});

const turnstileEnabled =
Boolean(import.meta.env["VITE_TURNSTILE_SITE_KEY"]);

function emptyBookingForm() {
  return {
    firstName: "",
    middleName: "",
    lastName: "",
    phone: "",
    email: "",
    motoBrand: "",
    motoModel: "",
    motoVariant: "",
    motoYear: String(new Date().getFullYear()),
    plateNumber: "",
    notes: "",
    wantsProducts: null as boolean | null,
  };
```

```
}

const bookingDraftSchema = z.object({
  form: z.object({
    firstName: z.string(),
    middleName: z.string(),
    lastName: z.string(),
    phone: z.string(),
    email: z.string(),
    motoBrand: z.string(),
    motoModel: z.string(),
    motoVariant: z.string(),
    motoYear: z.string(),
    plateNumber: z.string(),
    notes: z.string(),
    wantsProducts: z.boolean().nullable(),
  }),
  serviceIds: z.array(z.string()),
  date: z.string(),
  startTime: z.string(),
});
type BookingDraft = z.infer<typeof bookingDraftSchema>;

function parseBookingDraft(value: unknown) {
  const parsed = bookingDraftSchema.safeParse(value);
  return parsed.success ? parsed.data : null;
}
const BOOKING_PAGES = ["Booking details", "Review & submit"] as const;

export const Route = createFileRoute("/book")({
  validateSearch: searchSchema,
  head: () => ({
    meta: [
      { title: "Book a Service Appointment | Fake Rider Motorparts" },
      {
        name: "description",
        content:
          "Reserve your motorcycle service slot online. Choose your services and
schedule a convenient time, then save your reference code.",
      },
      { property: "og:title", content: "Book a Service Appointment | Fake Rider" },
      {
        property: "og:description",
        content: "Reserve your motorcycle service slot online in a few minutes.",
      },
```

```
    ],
  }),
  component: BookPage,
});

type Errors = Partial<{
  firstName: string;
  middleName: string;
  lastName: string;
  phone: string;
  email: string;
  motoBrand: string;
  motoModel: string;
  plateNumber: string;
  services: string;
  wantsProducts: string;
  date: string;
  startTime: string;
  schedule: string;
  terms: string;
  rescheduleReason: string;
}>;

const validationFieldOrder: (keyof Errors)[] = [
  "lastName",
  "firstName",
  "middleName",
  "phone",
  "email",
  "motoBrand",
  "motoModel",
  "plateNumber",
  "services",
  "wantsProducts",
  "rescheduleReason",
  "date",
  "startTime",
  "schedule",
  "terms",
];

const validationFocusTargets: Partial<Record<keyof Errors, string>> = {
  firstName: "#booking-first-name",
  middleName: "#booking-middle-name",
  lastName: "#booking-last-name",
```

```
  phone: "#booking-phone",
  email: "#booking-email",
  motoBrand: "#booking-moto-brand",
  motoModel: "#booking-moto-model",
  plateNumber: "#booking-plate-number",
  services: "#booking-services",
  wantsProducts: "#booking-products",
  rescheduleReason: "#reschedule-reason",
  date: "#booking-date button[data-day]:not([disabled])",
  startTime: "#booking-start-time button:not([disabled])",
  schedule: "#booking-schedule",
  terms: "#booking-terms",
};

function isDuplicateBookingMessage(message: string) {
  return /already (?:have|has) an appointment for/i.test(message);
}

function BookPage() {
  const book = useServerFn(createBooking);
  const availabilityFn = useServerFn(getAvailability);
  const rescheduleDetailsFn = useServerFn(getRescheduleDetails);
  const search = useSearch({ from: "/book" });

  const [form, setForm] = useState(emptyBookingForm);
  const [serviceIds, setServiceIds] = useState<string[]>(
    search.serviceId ? [search.serviceId] : [],
  );
  const [serviceCategory, setServiceCategory] = useState("all");
  const [serviceSearch, setServiceSearch] = useState("");
  const [date, setDate] = useState<string>(search.date ?? "");
  const [startTime, setStartTime] = useState<string>(search.startTime ?? "");
  const [terms, setTerms] = useState(false);
  const [bookingPage, setBookingPage] = useState<"details" |
"review">("details");
  const [errors, setErrors] = useState<Errors>({});
  const [focusRequest, setFocusRequest] = useState<keyof Errors | null>(null);
  const [multiDayContinuationOpen, setMultiDayContinuationOpen] =
useState(false);
  const [multiDayContinuationAccepted, setMultiDayContinuationAccepted] =
useState(false);
  const [duplicateBookingMessage, setDuplicateBookingMessage] =
useState<string | null>(null);
  const [result, setResult] = useState<{
    reference: string;
```

```
    total: number;
    date: string;
    startTime: string;
    continuationSegments?: Array<{ date: string; startTime: string;
durationMinutes: number }>;
  } | null>(null);
  const [turnstileToken, setTurnstileToken] = useState("");
  const [bookingRequestId, setBookingRequestId] = useState(() => randomId());
  const isReschedule = Boolean(search.reschedule && search.phone);
  const [rescheduleReason, setRescheduleReason] = useState("");
  const [reschedulePrefillReady, setReschedulePrefillReady] =
useState(!isReschedule);

  const rescheduleDetails = useQuery({
    queryKey: ["reschedule-details", search.reschedule, search.phone],
    queryFn: async () => {
      if (!search.reschedule || !search.phone) {
        throw new Error("Missing reschedule appointment details.");
      }
      return rescheduleDetailsFn({
        data: {
          reference: search.reschedule,
          phone: normalizePhilippineMobile(search.phone)!,
        },
      });
    },
    enabled: isReschedule,
    retry: false,
  });

  const rescheduleError =
    rescheduleDetails.data && !rescheduleDetails.data.ok
      ? rescheduleDetails.data.error
      : rescheduleDetails.isError
        ? "We could not load your original appointment. Please return to the
Appointment Tracker and try again."
        : null;
  const rescheduleReady =
    !isReschedule || (rescheduleDetails.data?.ok === true &&
reschedulePrefillReady);

  const services = useQuery({
    queryKey: ["services"],
    queryFn: async () => {
      const { data, error } = await supabase
```

```
        .from("services")
        .select("*")
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("sort_order");
      if (error) throw error;
      return Array.from(new Map((data ?? []).map((s) => [s.name.trim(),
s])).values());
    },
  });

  const shopSettings = useQuery({
    queryKey: ["public-shop-settings"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("shop_settings")
        .select("booking_terms,opening_time,closing_time")
        .eq("id", true)
        .maybeSingle();
      if (error) throw error;
      return data;
    },
  });

  useEffect(() => {
    if (!isReschedule || !rescheduleDetails.data?.ok) return;
    const appointment = rescheduleDetails.data.appointment;
    setForm({
      firstName: appointment.firstName,
      middleName: appointment.middleName,
      lastName: appointment.lastName,
      phone: appointment.phone,
      email: appointment.email,
      motoBrand: appointment.motoBrand,
      motoModel: appointment.motoModel,
      motoVariant: appointment.motoVariant,
      motoYear: appointment.motoYear,
      plateNumber: appointment.plateNumber,
      notes: appointment.notes,
      wantsProducts: appointment.wantsProducts,
    });
    setServiceIds(appointment.serviceIds);
    setServiceCategory("all");
    setDate("");
    setStartTime("");
```

```
    setBookingPage("details");
    setReschedulePrefillReady(true);
  }, [isReschedule, rescheduleDetails.data]);

  const motorcycleCatalog = useQuery({
    queryKey: ["motorcycle-catalog"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("motorcycle_catalog")
        .select("brand,model")
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("brand")
        .order("model");
      if (error) throw error;
      return Array.from(
        new Map((data ?? []).map((m) => [`${m.brand ?? ""}:${m.model.trim()}`,
m])).values(),
      );
    },
  });

  const bookableMotorcycles = useMemo(
    () =>
      (motorcycleCatalog.data ?? []).filter((motorcycle) => {
        return motorcycle.brand && motorcycle.model;
      }),
    [motorcycleCatalog.data],
  );

  const brands = useMemo(() => {
    const set = new Set<string>();
    bookableMotorcycles.forEach((p) => p.brand && set.add(p.brand));
    return Array.from(set).sort();
  }, [bookableMotorcycles]);

  const catalogUnavailable =
    motorcycleCatalog.isError || (!motorcycleCatalog.isLoading && brands.length
=== 0);

  const modelsForBrand = useMemo(() => {
    const seen = new Set<string>();
    const list = bookableMotorcycles.filter((p) => p.brand === form.motoBrand);
    return list
      .filter((p) => {
```

```
        if (seen.has(p.model)) return false;
        seen.add(p.model);
        return true;
      })
      .map((p) => ({
        value: p.model,
        label: p.model.replace(new RegExp(`^${p.brand} `), ""),
      }))
      .sort((a, b) => a.label.localeCompare(b.label, undefined, { sensitivity: "base"
}));
  }, [bookableMotorcycles, form.motoBrand]);

  const motorcycleSelectionReady = Boolean(form.motoBrand &&
form.motoModel);

  const availability = useQuery<Availability>({
    queryKey: [
      "availability",
      serviceIds,
      form.motoBrand,
      form.motoModel,
      isReschedule,
      serviceIds.length > 1,
    ],
    queryFn: () =>
      availabilityFn({
        data: {
          days: 45,
          serviceIds,
          rescheduling: isReschedule,
          allowMultiDayContinuation: !isReschedule && serviceIds.length > 1,
          motorcycle: { brand: form.motoBrand, model: form.motoModel },
          ...(search.reschedule && search.phone
            ? {
                rescheduleReference: search.reschedule,
                reschedulePhone: normalizePhilippineMobile(search.phone)!,
              }
            : {}),
        },
      }),
    enabled: serviceIds.length > 0 && motorcycleSelectionReady,
  });

  const dates = useMemo(
    () => (availability.data ? computeAvailableDates(availability.data) : []),
```

```
    [availability.data],
  );

  const slots = useMemo(
    () => (availability.data && date ? computeAvailableSlots(availability.data, date)
: []),
    [availability.data, date],
  );

  const availableDateSet = useMemo(() => new Set(dates), [dates]);
  const fullyBookedDates = useMemo(
    () => (availability.data ? computeFullyBookedDates(availability.data) : []),
    [availability.data],
  );

  const availabilityEnabled = serviceIds.length > 0 && motorcycleSelectionReady;
  const availabilityLoading =
    availabilityEnabled && (availability.isPending || availability.isFetching);
  const availabilityError = availability.data?.error;

  const selectedServices = (services.data ?? []).filter((s) =>
serviceIds.includes(s.id));
  const serviceCategories = useMemo(
    () =>
      Array.from(
        new Set((services.data ?? []).map((service) =>
service.category).filter(Boolean)),
      ).sort(),
    [services.data],
  );
  const normalizedServiceSearch = serviceSearch.trim().toLowerCase();
  const filteredServices = (services.data ?? []).filter(
    (service) =>
      (serviceCategory === "all" || service.category === serviceCategory) &&
      (!normalizedServiceSearch ||
service.name.toLowerCase().includes(normalizedServiceSearch)),
  );
  const serviceEstimates =
    availability.data?.serviceEstimates ??
    selectedServices.map((service) => ({
      id: service.id,
      name: service.name,
      price: Number(service.price),
      durationMinutes: service.duration_minutes ?? 60,
      pricingSource: "small_bike" as const,
```

```
    }));
  const estimatedServices = serviceIds
    .map((serviceId) => serviceEstimates.find((service) => service.id ===
serviceId))
    .filter((service): service is NonNullable<typeof service> => Boolean(service));
  const total = estimatedServices.reduce((sum, service) => sum + service.price,
0);
  const customerFullName = [form.firstName, form.middleName, form.lastName]
    .map((part) => part.trim())
    .filter(Boolean)
    .join(" ");
  const totalDuration = estimatedServices.reduce(
    (sum, service) => sum + service.durationMinutes,
    0,
  );
  const operatingHours =
    availability.data?.operatingHours ??
    (shopSettings.data
      ? {
          openingTime: shopSettings.data.opening_time
            ? String(shopSettings.data.opening_time).slice(0, 5)
            : DEFAULT_BOOKING_HOURS.openingTime,
          closingTime: shopSettings.data.closing_time
            ? String(shopSettings.data.closing_time).slice(0, 5)
            : DEFAULT_BOOKING_HOURS.closingTime,
        }
      : DEFAULT_BOOKING_HOURS);
  const serviceStepOverflowMinutes =
    !isReschedule && serviceIds.length > 1
      ? bookingDurationOverflowMinutes(operatingHours.openingTime,
totalDuration, operatingHours)
      : 0;
  const needsServiceStepMultiDayConfirmation = serviceStepOverflowMinutes >
0;
  const multiDayOverflowMinutes =
    !isReschedule && date && startTime && serviceIds.length > 1
      ? bookingDurationOverflowMinutes(startTime, totalDuration, operatingHours)
      : 0;
  const needsMultiDayContinuationConfirmation = multiDayOverflowMinutes > 0;
  const bookingTerms = shopSettings.data?.booking_terms ||
DEFAULT_BOOKING_TERMS;
  // Prices depend on the motorcycle's size category, so only show a total once
  // the server has priced the selected services for the chosen motorcycle.
  const pricingReady = Boolean(availability.data?.serviceEstimates) &&
!availabilityError;
```

```
  const selectedModelLabel =
    modelsForBrand.find((model) => model.value === form.motoModel)?.label ??
form.motoModel;
  const sectionComplete = {
    personal:
      isReschedule ||
      (form.lastName.trim().length >= 2 &&
        form.firstName.trim().length >= 2 &&
        normalizePhilippineMobile(form.phone) !== null &&
        /^\S+@\S+\.\S+$/.test(form.email.trim())),
    motorcycle: motorcycleSelectionReady && form.plateNumber.trim().length >=
2,
    services: serviceIds.length > 0 && form.wantsProducts !== null,
    schedule: Boolean(date && startTime) && (!isReschedule ||
Boolean(rescheduleReason.trim())),
  };

  function clearErrors(...fields: (keyof Errors)[]) {
    setErrors((current) => {
      if (!fields.some((field) => current[field])) return current;
      const next = { ...current };
      for (const field of fields) delete next[field];
      return next;
    });
  }

  function updateForm<K extends keyof typeof form>(field: K, value: (typeof
form)[K]) {
    setForm((current) => ({ ...current, [field]: value }));
    if (field in validationFocusTargets) clearErrors(field as keyof Errors);
  }

  // Restore an unfinished booking once the page is open in the browser. A
  // service, date or time in the link (from "Book this service") wins.
  const [draftLoaded, setDraftLoaded] = useState(false);
  const [draftRestored, setDraftRestored] = useState(false);
  useEffect(() => {
    if (draftLoaded) return;
    const draft = isReschedule ? null : loadBookingDraft(parseBookingDraft);
    if (draft) {
      setForm(draft.form);
      if (!search.serviceId) setServiceIds(draft.serviceIds);
      if (!search.date) {
        setDate(draft.date);
        setStartTime(draft.startTime);
```

```
      }
      setDraftRestored(true);
    }
    setDraftLoaded(true);
  }, [draftLoaded, isReschedule, search.serviceId, search.date]);

  // Keep the draft up to date while the customer fills in the form.
  useEffect(() => {
    if (!draftLoaded || isReschedule || result) return;
    const draft: BookingDraft = { form, serviceIds, date, startTime };
    const hasInput =
      Object.entries(form).some(
        ([field, value]) => field !== "motoYear" && value !== "" && value !== null,
      ) ||
      serviceIds.length > 0 ||
      date !== "";
    if (hasInput) saveBookingDraft(draft);
    else clearBookingDraft();
  }, [draftLoaded, isReschedule, result, form, serviceIds, date, startTime]);

  function startOver() {
    clearBookingDraft();
    setDraftRestored(false);
    setForm(emptyBookingForm());
    setServiceIds([]);
    setServiceCategory("all");
    setServiceSearch("");
    resetSchedule();
    setTerms(false);
    setErrors({});
    setDuplicateBookingMessage(null);
    setBookingPage("details");
  }

  function resetSchedule() {
    setDate("");
    setStartTime("");
    setMultiDayContinuationAccepted(false);
  }

  function toggleService(serviceId: string) {
    const checked = serviceIds.includes(serviceId);
    if (!checked && serviceIds.length >=
MAX_BOOKING_SERVICE_SELECTIONS) {
      setErrors((current) => ({
```

```
        ...current,
        services: `You can select up to
${MAX_BOOKING_SERVICE_SELECTIONS} services per appointment.`,
      }));
      return;
    }
    setServiceIds((prev) =>
      checked ? prev.filter((id) => id !== serviceId) : [...prev, serviceId],
    );
    clearErrors("services");
    resetSchedule();
  }

  function clearServices() {
    setServiceIds([]);
    resetSchedule();
  }

  useEffect(() => {
    if (!focusRequest) return;

    const target = validationFocusTargets[focusRequest];
    if (!target) return;

    // Let React show the appropriate booking page before trying to focus its input.
    const timer = window.setTimeout(() => {
      const element = document.querySelector<HTMLElement>(target);
      element?.scrollIntoView({ behavior: "smooth", block: "center" });
      element?.focus({ preventScroll: true });
      setFocusRequest(null);
    }, 0);

    return () => window.clearTimeout(timer);
  }, [bookingPage, focusRequest]);

  useEffect(() => {
    if (!availability.data || !date) return;

    if (!availableDateSet.has(date)) {
      setDate("");
      setStartTime("");
      return;
    }

    if (startTime) {
```

```
    const selectedSlot = computeAvailableSlots(availability.data, date).find(
      (slot) => slot.startTime === startTime,
    );
    if (!selectedSlot || selectedSlot.disabled) setStartTime("");
  }
}, [availability.data, availableDateSet, date, startTime]);

const mutation = useMutation({
  mutationFn: async () => {
    if (form.wantsProducts === null) {
      throw new Error("Choose whether you would like to avail shop products.");
    }
    const submittedSchedule = { date, startTime };
    const res = await book({
      data: {
        firstName: form.firstName.trim(),
        middleName: form.middleName.trim(),
        lastName: form.lastName.trim(),
        phone: normalizePhilippineMobile(form.phone)!,
        email: form.email.trim().toLowerCase(),
        motoBrand: form.motoBrand.trim(),
        motoModel: form.motoModel.trim(),
        motoVariant: form.motoVariant.trim(),
        motoYear: Number(form.motoYear),
        plateNumber: form.plateNumber.trim(),
        serviceIds,
        date,
        startTime,
        notes: form.notes.trim(),
        wantsProducts: form.wantsProducts,
        turnstileToken,
        idempotencyKey: bookingRequestId,
        termsAccepted: true as const,
        multiDayContinuationAccepted,
        ...(isReschedule ? { rescheduleReference: search.reschedule } : {}),
        ...(isReschedule ? { rescheduleReason: rescheduleReason.trim() } : {}),
      },
    });
    return { res, submittedSchedule };
  },
  onSuccess: ({ res, submittedSchedule }) => {
    if (!res.ok) {
      if (!isReschedule && isDuplicateBookingMessage(res.error)) {
        setDuplicateBookingMessage(res.error);
        setErrors(({ schedule: _, ...current }) => current);
```

```
          setDate("");
          setStartTime("");
          setTurnstileToken("");
          setBookingRequestId(randomId());
          availability.refetch();
          return;
        }
        setErrors((current) => ({ ...current, schedule: res.error }));
        setTurnstileToken("");
        setBookingRequestId(randomId());
        availability.refetch();
        return;
      }
      clearBookingDraft();
      setResult(
        "continuationSegments" in res
          ? {
              reference: res.reference,
              total: res.total,
              date: submittedSchedule.date,
              startTime: submittedSchedule.startTime,
              continuationSegments: res.continuationSegments,
            }
          : {
              reference: res.reference,
              total: res.total,
              date: submittedSchedule.date,
              startTime: submittedSchedule.startTime,
            },
      );
      window.scrollTo({ top: 0, behavior: "smooth" });
    },
    onError: (err: Error) => {
      if (!isReschedule && isDuplicateBookingMessage(err.message)) {
        setDuplicateBookingMessage(err.message);
        setErrors(({ schedule: _, ...current }) => current);
        setDate("");
        setStartTime("");
        return;
      }
      const msg = err.message.toLowerCase();
      let scheduleError: string;
      if (msg.includes("network") || msg.includes("fetch") ||
msg.includes("connect")) {
        scheduleError = "Network error. Please check your connection and try
```

```
again.";
      } else if (msg.includes("timeout")) {
        scheduleError = "The request timed out. Please try again.";
      } else if (msg.includes("validation") || msg.includes("invalid")) {
        scheduleError = "Some fields have invalid values. Please review the form.";
      } else {
        scheduleError =
          "We could not submit your booking. Please review your details and try
again.";
      }
      setErrors((current) => ({ ...current, schedule: scheduleError }));
      availability.refetch();
    },
  });

  function validationErrors(steps: number[]) {
    const e: Errors = {};
    if (isReschedule && !rescheduleReady) {
      e.schedule = rescheduleError ?? "Loading the original appointment details.";
      return e;
    }
    if (steps.includes(1)) {
      if (form.lastName.trim().length < 2) e.lastName = "Enter your last name.";
      if (form.firstName.trim().length < 2) e.firstName = "Enter your first name.";
      if (!normalizePhilippineMobile(form.phone)) e.phone =
PHONE_VALIDATION_MESSAGE;
      if (!isReschedule && !/^\S+@\S+\.\S+$/.test(form.email.trim())) {
        e.email = "Enter a valid email address.";
      }
    }
    if (steps.includes(2)) {
      if (!form.motoBrand.trim()) e.motoBrand = "Required";
      if (!form.motoModel.trim()) e.motoModel = "Required";
      if (!isReschedule && catalogUnavailable) {
        e.motoBrand = "The motorcycle catalog is unavailable. Please try again
shortly.";
      }
      if (
        !isReschedule &&
        !catalogUnavailable &&
        form.motoBrand &&
        !brands.includes(form.motoBrand)
      ) {
        e.motoBrand = "Select a brand from the motorcycle catalog.";
      }
```

```
      if (
        !isReschedule &&
        !catalogUnavailable &&
        form.motoModel &&
        !modelsForBrand.some((model) => model.value === form.motoModel)
      ) {
        e.motoModel = "Select a model from the motorcycle catalog.";
      }
      if (form.plateNumber.trim().length < 2) e.plateNumber = "Required";
    }
    if (steps.includes(3) && serviceIds.length === 0) e.services = "Select at least
one service.";
    if (steps.includes(3) && serviceIds.length >
MAX_BOOKING_SERVICE_SELECTIONS) {
      e.services = `Select up to ${MAX_BOOKING_SERVICE_SELECTIONS}
services per appointment.`;
    }
    if (steps.includes(4) && !date) e.date = "Select your preferred date.";
    if (steps.includes(4) && date && !startTime) e.startTime = "Select your
preferred time.";
    if (steps.includes(3) && form.wantsProducts === null) {
      e.wantsProducts = "Choose Yes or No for shop products.";
    }
    if (steps.includes(4) && availabilityLoading) {
      e.schedule = "Availability is still loading. Please wait a moment.";
    }
    if (steps.includes(4) && (availability.isError || availabilityError)) {
      e.schedule = availabilityError ?? "We could not verify availability. Please try
again.";
    }
    if (steps.includes(4) && isReschedule && !rescheduleReason.trim()) {
      e.rescheduleReason = "Tell us why you need to reschedule.";
    }
    if (steps.includes(5) && !terms) e.terms = "You must accept the terms and
conditions.";
    return e;
  }

  function validateBooking() {
    const e = validationErrors([1, 2, 3, 4, 5]);
    return showValidationErrors(e);
  }

  function showValidationErrors(e: Errors) {
    setErrors(e);
```

```
    const firstInvalidField = validationFieldOrder.find((field) => e[field]);
    if (!firstInvalidField) return true;

    setBookingPage(firstInvalidField === "terms" ? "review" : "details");
    setFocusRequest(firstInvalidField);
    return false;
  }

  function continueToReview() {
    const e = validationErrors([1, 2, 3, 4]);
    if (!showValidationErrors(e)) return;
    const requiresMultiDayConfirmation =
      needsServiceStepMultiDayConfirmation ||
needsMultiDayContinuationConfirmation;
    if (requiresMultiDayConfirmation && !multiDayContinuationAccepted) {
      setMultiDayContinuationOpen(true);
      return;
    }
    setBookingPage("review");
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  function continueWithMultiDayBooking() {
    setMultiDayContinuationAccepted(true);
    setMultiDayContinuationOpen(false);
    setBookingPage("review");
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  function returnToBookingDetails() {
    setErrors({});
    setBookingPage("details");
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  function submitBooking() {
    if (!rescheduleReady || !validateBooking()) return;
    mutation.mutate();
  }

  /** Return from the review page to a specific part of the booking form. */
  function editBookingSection(field: keyof Errors) {
    setErrors({});
    setBookingPage("details");
    setFocusRequest(field);
```